Data Structure and Algorithm

Chapter 3: Stacks and Queuens

Prepared By: Guta T.

AmiT 2022

Stacks

- A stack is called a last-in-first-out (LIFO) collection.
- This means that the last thing we added (pushed) is the first thing that gets pulled (popped) off .
- A stack is a sequence of items that are accessible at only one end of the sequence.
- Stacks is LIFO structure and physically can be implemented as an array or as a linked list.
- Stack, when implemented as an array is functionally same as any other array except that here, adding an element and deletion is done from the same direction just like a pile of books.

Stacks

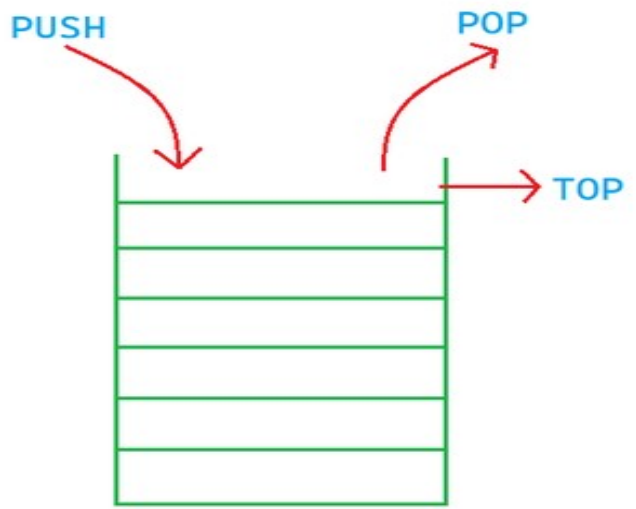
- Mainly the following three basic operations are performed in the stack:
- **Push:** Adds an item in the stack. If the stack is full, then it is said to be an Overflow condition.
- **Pop:** Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an Underflow condition.
- **Peek or Top:** Returns the top element of the stack.
- **isEmpty:** Returns true if the stack is empty, else false.

Stacks

- The fundamental operations that can be performed on stack are **PUSH and POP.**
- When element is added on the stack top is called **PUSH.**
- When Data is removed from the stack top, the operation is called **POP.**

Stack

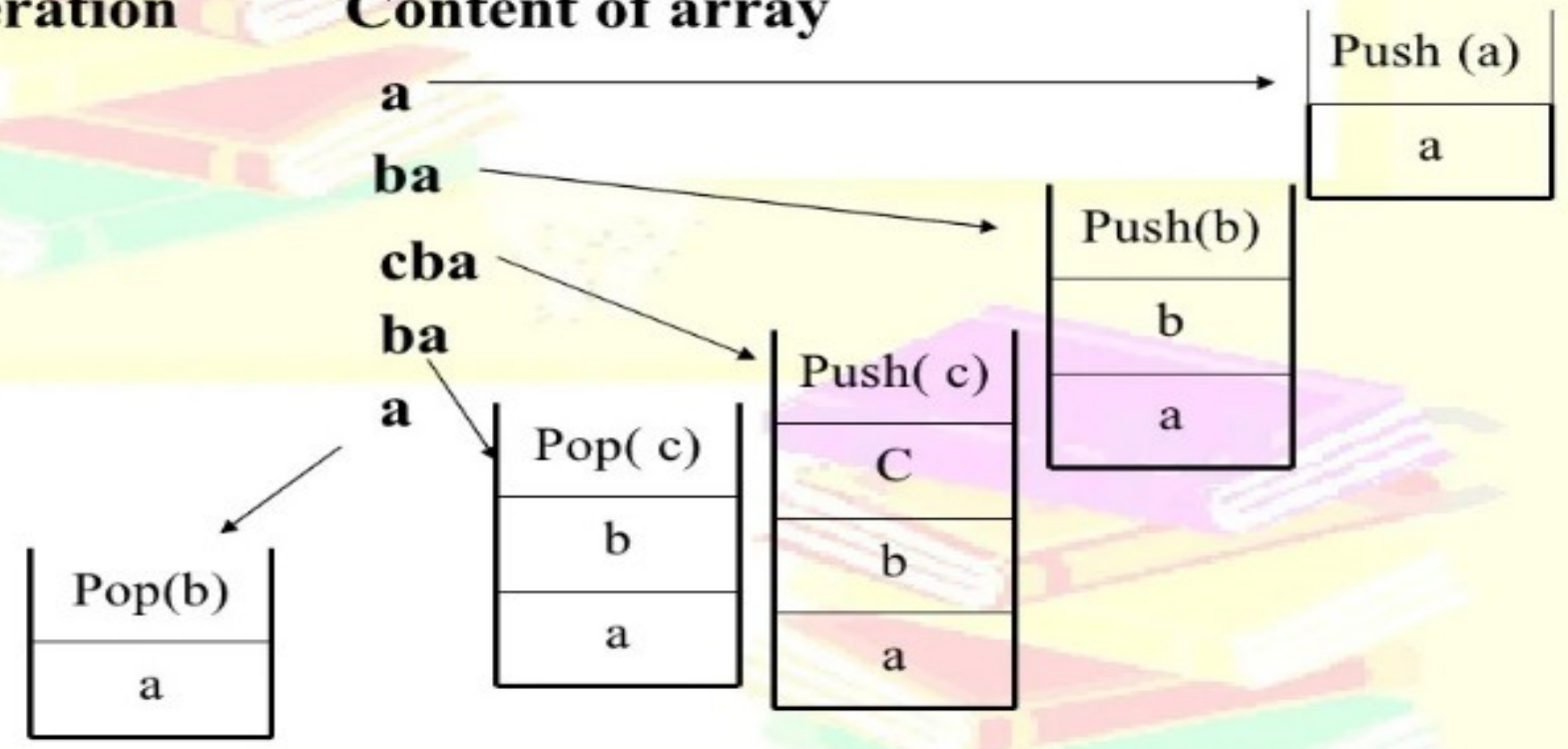
Insertion & Deletion happens on the same end.



Stacks Implementation:

The stack operation can be explained as follows:

Stack operation	Content of array
Push(a)	a
Push(b)	ba
Push(c)	cba
Pop()	ba
Pop()	a



Stacks Implementation:

There are two ways to implement a stack:

- **Using array**

Pros: Easy to implement. Memory is saved as pointers are not involved.

Cons: It is not dynamic. It doesn't grow and shrink depending on needs at runtime.

- **Using linked list**

Pros: The linked list implementation of a stack can grow and shrink according to the needs at runtime.

Cons: Requires extra memory due to involvement of pointers.

Stacks implementation Using array

Algorithm

push(item)

Begin

increase the top pointer by 1

insert item into the location top

End

Algorithm

pop()

Begin

item = top element from stack

reduce top pointer by 1

return item

End

Stacks implementation Using array

Algorithm

peek()

Begin

  item = top element from stack

  return item

End

- **Example code…..**

```cpp
/* C++ program to implement basic stack operations using array
By Guta */
#include <bits/stdc++.h>

using namespace std;

#define MAX 1000

class Stack {
    int top;

public:
    int a[MAX]; // Maximum size of Stack

    Stack() { top = -1; }
    bool push(int x);
    int pop();
    int peek();
    bool isEmpty();
};

bool Stack::push(int x)
{
    if (top >= (MAX - 1)) {
        cout << "Stack Overflow";
        return false;
    }
    else {
        a[++top] = x;
        cout << x << " pushed into stack\n";
        return true;
    }
}

int Stack::pop()
{
    if (top < 0) {
        cout << "Stack Underflow";
        return 0;
    }
    else {
        int x = a[top--];
        return x;
    }
}
int Stack::peek()
{
    if (top < 0) {
        cout << "Stack is Empty";
        return 0;
    }
    else {
        int x = a[top];
        return x;
    }
}

bool Stack::isEmpty()
{
    return (top < 0);
}
```

```
63  // Driver program to test above functions
64  int main()
65  {
66      class Stack s;
67      s.push(10);
68      s.push(20);
69      s.push(30);
70      cout << s.pop() << " Popped from stack\n";
71      //print all elements in stack :
72      cout<<"Elements present in stack : ";
73      while(!s.isEmpty())
74      {
75          // print top element in stack
76          cout<<s.peek()<<" ";
77          // remove top element in stack
78          s.pop();
79      }
80
81      return 0;
82  }
83
```

Stacks implementation Using linked list

```cpp
/* C++ program for linked list implementation of stack
using Linked list By: Guta, AMU*/
#include <bits/stdc++.h>
using namespace std;

// A structure to represent a stack
class StackNode {
public:
    int data;
    StackNode* next;
};

StackNode* newNode(int data)
{
    StackNode* stackNode = new StackNode();
    stackNode->data = data;
    stackNode->next = NULL;
    return stackNode;
}

int isEmpty(StackNode* root)
{
    return !root;
}

void push(StackNode** root, int data)
{
    StackNode* stackNode = newNode(data);
    stackNode->next = *root;
    *root = stackNode;
    cout << data << " pushed to stack\n";
}

int pop(StackNode** root)
{
    if (isEmpty(*root))
        return INT_MIN;
    StackNode* temp = *root;
    *root = (*root)->next;
    int popped = temp->data;
    free(temp);

    return popped;
}
```

```
46  int peek(StackNode* root)
47  {
48      if (isEmpty(root))
49          return INT_MIN;
50      return root->data;
51  }
52
53  // Driver code
54  int main()
55  {
56      StackNode* root = NULL;
57      push(&root, 10);
58      push(&root, 20);
59      push(&root, 30);
60      cout << pop(&root) << " popped from stack\n";
61
62      cout << "Top element is " << peek(root) << endl;
63
64      cout<<"Elements present in stack : ";
65      //print all elements in stack :
66      while(!isEmpty(root))
67      {// print top element in stack
68          cout<<peek(root)<<" ";
69          // remove top element in stack
70          pop(&root);
71      }
72      return 0;
73  }
74
```

Queues

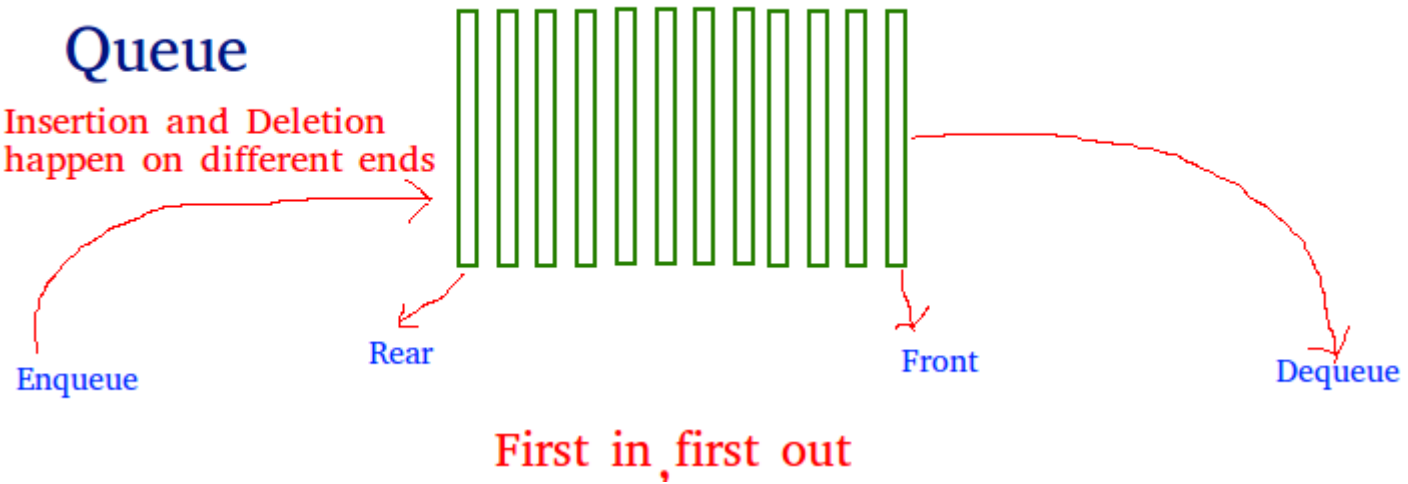
The queue is a basic data structure just like a stack.

- Queue is FIFO (First In First Out) structure and physically can be implemented as an array or as a linked list.
- Queue, when implemented as an array is functionally same as any other array except that here, adding an element and deletion is done from the one direction and deletion from other just like any queue of peoples.



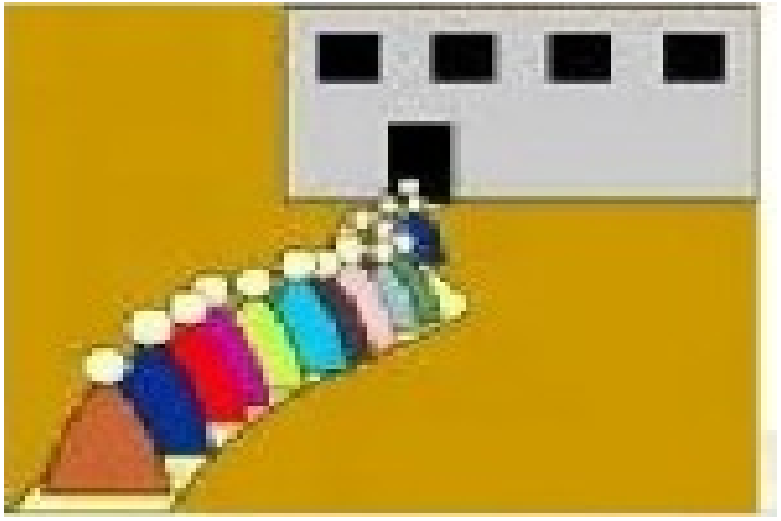
Queuens

- A good example of a queue is any queue of consumers for a resource where the consumer that came first is served first.
- The difference between stacks and queues is in removing.
- In a stack we remove the item the most recently added
- I in a queue, we remove the item the least recently added.



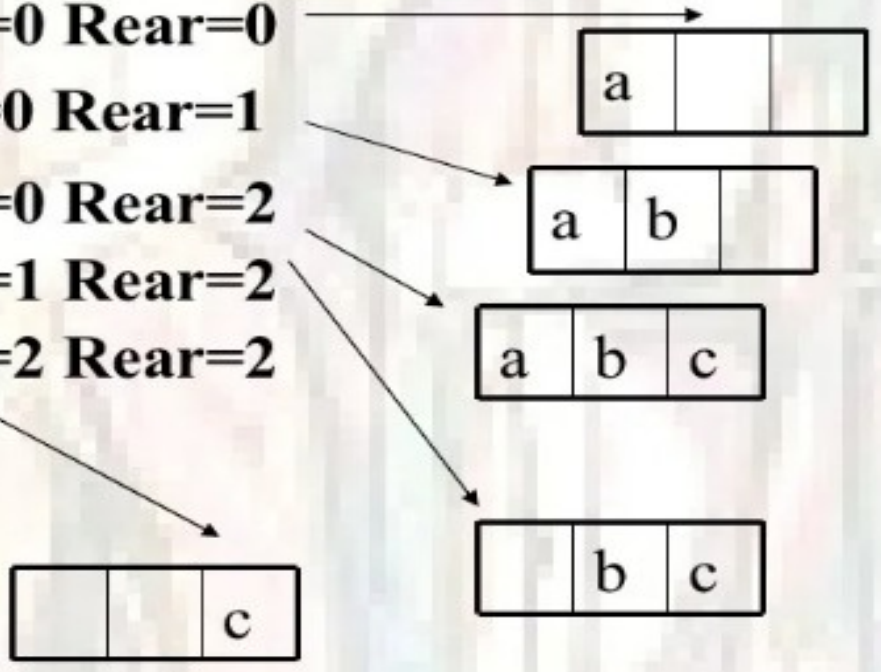
Basic operation and implementation of QUEUE

- The fundamental operations that can be performed on Queue are Insert and Delete.
- When element is added on the Queue Front is called Insert.
- When Data is removed from the Queue Rear, the operation is called Delete.
- Basic operation in queuens
  - Creation of Queue
  - Check for empty Queue
  - Check for full Queue
  - Add element in Queue
  - Delete element in Queue
  - Print Queue



The Queue operation can be explained as follows:

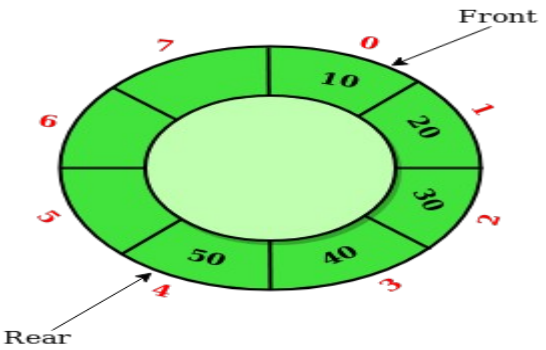
Queue operation	Content of array
Insert(a)	Front=0 Rear=0
Insert(b)	Front=0 Rear=1
Insert(c)	Front=0 Rear=2
Delete()	Front=1 Rear=2
Delete()	Front=2 Rear=2



If we try to insert
Overflow occurs
Though first two cells are empty

Circular array implementation of enqueue and dequeue operations

- **Circular Queue** is a linear data structure in which the operations are performed based on FIFO principle and the last position is connected back to the first position to make a circle. It is also called '**Ring Buffer**'.



- In a normal Queue, we can insert elements until queue becomes full. But once queue becomes full, we can not insert the next element even if there is a space in front of queue.

enQueue(value) This function is used to insert an element into the circular queue. In a circular queue, the new element is always inserted at Rear position.

1.Check whether queue is Full – Check ((rear == SIZE-1 && front == 0) || (rear == front-1)).

2.If it is full then display Queue is full. If queue is not full then, check if (rear == SIZE – 1 && front != 0) if it is true then set rear=0 and insert element

deQueue() This function is used to delete an element from the circular queue. In a circular queue, the element is always deleted from front position.

1.Check whether queue is Empty means check (front==-1).

2.If it is empty then display Queue is empty. If queue is not empty then step 3

3.Check if (front==rear) if it is true then set front=rear= -1 else check if (front==size-1), if it is true then set front=0 and return the element.

Example Code

```cpp
// C++ program for insertion and
// deletion in Circular Queue By:Guta
#include<bits/stdc++.h>
using namespace std;

class Queue
{// Initialize front and rear
    int rear, front;
    // Circular Queue
    int size;
    int *arr;
public:
    Queue(int s)
    {
    front = rear = -1;
    size = s;
    arr = new int[s];
    }

    void enQueue(int value);
    int deQueue();
    void displayQueue();
};

```

Priority Queue

Priority Queue is an abstract data type, which is similar to a queue, however, in the priority queue, every element has some priority. The priority of the elements in a priority queue determines the order in which elements are removed from the priority queue. Therefore all the elements are either arranged in an ascending or descending order.

So, a priority Queue is an extension of the queue with the following properties.

- Every item has a priority associated with it.
- An element with high priority is dequeued before an element with low priority.
- If two elements have the same priority, they are served according to their order in the queue.